

## `GridSearchCV`와 `RandomizedSearchCV`는

하이퍼파라미터 튜닝을 위한 사이킷런(Scikit-learn) 라이브러리의 두 가지 중요한 도구로, 모델의 성능을 최적화하기 위해 사용됩니다. 두 방법은 교차 검증을 통해 여러 하이퍼파라미터 조합을 평가하고 가장 좋은 성능을 내는 조합을 찾습니다. 하지만 탐색 방식, 효율성, 계산 비용 등의 차이로 인해 사용 목적과 상황에 따라 선택해야 합니다.

### 1. 탐색 방식 (Search Strategy)

#### - `GridSearchCV`

- 모든 가능한 하이퍼파라미터 조합을 전부 탐색합니다.
- 각 하이퍼파라미터에 대해 사용자가 미리 지정한 값들의 그리드(grid)를 생성하고, 그리드의 각 조합에 대해 교차 검증을 실행하여 성능을 평가합니다.
- 전수조사 방식(exhaustive search)으로, 지정된 하이퍼파라미터 값들을 전부 조합해서 시도합니다.
- 예를 들어, 두 개의 하이퍼파라미터 `param1 = [1, 10]`과 `param2 = [0.01, 0.1, 1]`이 있을 때, GridSearchCV는  $2 \times 3 = 6$ 가지 조합을 모두 평가합니다.

#### - `RandomizedSearchCV`

- 무작위로 하이퍼파라미터 조합을 시도합니다.
- 각 하이퍼파라미터의 값들로부터 랜덤하게 조합을 샘플링하여, 설정된 횟수(`n_iter`)만큼 탐색합니다.
- 부분 탐색 방식으로, 전체 조합 중 일부를 무작위로 선택하여 탐색하므로, 전체 경우의 수를 모두 시도하지 않습니다.
- 위와 같은 예시에서, `n_iter=4`로 설정하면 6개의 조합 중 4개의 조합을 랜덤으로 선택해 평가합니다.

### 2. 탐색 공간 (Search Space)

#### - `GridSearchCV`

- 미리 정해진 값들의 모든 조합을 탐색하기 때문에, 탐색 공간은 고정된 격자(grid) 형태입니다.

- 각 하이퍼파라미터의 값이 많아질수록, 가능한 조합의 수가 기하급수적으로 증가하여 매우 큰 탐색 공간이 될 수 있습니다.
- 명확하게 값의 범위가 설정된 경우나, 작은 범위에서 최적의 파라미터를 찾고자 할 때 적합합니다.

- `RandomizedSearchCV`

- 연속적인 값이나 넓은 범위를 탐색할 수 있습니다. 예를 들어, 파라미터 값의 범위가 크거나 미리 정의되지 않은 경우에도 활용할 수 있습니다.
- 값의 범위만 설정해두면 그 범위 내에서 무작위로 값을 선택하여 탐색하기 때문에, 넓은 탐색 공간에서도 효과적입니다.
- 값이 연속적이거나, 탐색 공간이 매우 넓을 때 적합합니다.

### 3. 계산 비용 (Computational Cost)

- `GridSearchCV`

- 모든 조합을 시도하기 때문에, 조합의 수가 많아지면 계산 비용이 매우 높아질 수 있습니다.
- 예를 들어, 두 개의 파라미터에 각각 10개의 값을 지정한 경우,  $10 \times 10 = 100$ 개의 조합을 평가해야 합니다.
- 데이터셋이 크거나, 모델이 복잡한 경우에는 매우 많은 시간이 소요될 수 있습니다.

- `RandomizedSearchCV`

- 탐색할 조합의 수를 임의로 설정할 수 있기 때문에 계산 비용이 낮습니다.
- `n\_iter` 매개변수를 사용하여 탐색할 조합의 수를 제한할 수 있으며, 이를 통해 계산 비용을 효율적으로 제어할 수 있습니다.
- 모든 조합을 시도하지 않기 때문에, 대규모 데이터나 복잡한 모델에서도 시간과 자원을 절약할 수 있습니다.

### 4. 효율성 (Efficiency)

- `GridSearchCV`

- 모든 조합을 시도하기 때문에 최적의 파라미터를 놓칠 가능성이 거의

없습니다. 탐색 공간 안에서 가장 성능이 좋은 하이퍼파라미터 조합을 찾을 수 있습니다.

- 그러나 탐색해야 할 조합이 많아지면, 시간이 많이 걸리고 불필요한 조합까지 시도하게 되는 비효율적인 측면이 있습니다.

- `RandomizedSearchCV`

- 무작위로 선택한 조합만 평가하기 때문에, 항상 최적의 하이퍼파라미터를 찾을 수는 없을 수 있습니다.
- 그러나 탐색 범위가 넓거나 시간적 제약이 있을 경우, 더 적은 조합만으로도 효과적으로 좋은 성능을 내는 파라미터를 찾을 가능성이 높습니다.
- 계산 시간을 크게 절약할 수 있어, 실용적인 상황에서 매우 효율적입니다.

## 5. 유연성 (Flexibility)

- `GridSearchCV`

- 하이퍼파라미터 값들이 고정된 값들로 설정되어야 하므로, 하이퍼파라미터 공간을 세밀하게 조절하기 어렵습니다.
- 대규모 하이퍼파라미터 공간을 탐색할 때는 비효율적일 수 있습니다.

- `RandomizedSearchCV`

- 하이퍼파라미터 값들을 확률 분포로 설정할 수 있어, 유연하게 탐색 공간을 정의할 수 있습니다.
- 예를 들어, 연속적인 범위에서 하이퍼파라미터 값을 무작위로 선택할 수 있기 때문에, 보다 넓은 범위를 효율적으로 탐색할 수 있습니다.

## 6. 장단점 비교

- `GridSearchCV`

- 장점:
  - 모든 조합을 시도하므로, 탐색 범위 내에서 최적의 하이퍼파라미터 조합을 찾을 수 있습니다.

- 단점:
  - 계산 비용이 매우 높으며, 불필요한 조합까지 모두 시도하여 비효율적일 수 있습니다.
  - 탐색 공간이 넓어질수록 시간과 자원이 기하급수적으로 증가합니다.

- `RandomizedSearchCV`

- 장점:
  - 계산 비용이 낮고, 효율적입니다. 탐색할 조합의 수를 제어할 수 있기 때문에 빠르게 좋은 성능의 하이퍼파라미터를 찾을 수 있습니다.
  - 탐색 공간이 매우 넓어도 적은 조합만 탐색하여 시간을 절약할 수 있습니다.
- 단점:
  - 무작위로 선택된 조합만 평가하기 때문에 최적의 파라미터를 놓칠 가능성이 있습니다.

## 7. 사용 예시

### ● `GridSearchCV` 예시:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
```

```
# 하이퍼파라미터 그리드 정의
```

```
param_grid = {
    'n_estimators': [10, 50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}
```

```
# 모델 정의
```

```
rf = RandomForestClassifier()
```

```
# GridSearchCV 실행
```

```
grid_search = GridSearchCV(rf, param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

```
# 최적의 파라미터 출력
```

```
print("Best parameters:", grid_search.best_params_)
```

- **RandomizedSearchCV 예시:**

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
import numpy as np
```

```
# 하이퍼파라미터 범위 정의
```

```
param_dist = {
    'n_estimators': np.arange(10, 200, 10),
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': np.arange(2, 11)
}
```

```
# 모델 정의
```

```
rf = RandomForestClassifier()
```

```
# RandomizedSearchCV 실행
```

```
random_search = RandomizedSearchCV(rf, param_dist, n_iter=10, cv=5)
random_search.fit(X_train, y_train)
```

```
# 최적의 파라미터 출력
```

```
print("Best parameters:", random_search.best_params_)
```

## 결론

- 'GridSearchCV'는 작은 탐색 공간이나 모든 가능한 조합을 시도해야 할 때

적합한 방법입니다. 하지만, 탐색 범위가 넓어지면 비효율적일 수 있습니다.

## 결론

- `GridSearchCV`는 작은 탐색 공간에서 모든 가능한 조합을 시도하여 최적의 하이퍼파라미터를 찾는 데 적합합니다. 다만, 탐색 공간이 넓어질수록 계산 비용이 기하급수적으로 증가하므로, 비교적 적은 조합일 때 사용하는 것이 효율적입니다.

- `RandomizedSearchCV`는 탐색 공간이 넓거나 계산 자원이 제한된 상황에서 효율적입니다. 무작위로 하이퍼파라미터 조합을 시도하기 때문에, 시간과 자원을 절약하면서도 좋은 성능을 내는 하이퍼파라미터를 찾을 수 있습니다. 최적의 조합을 항상 보장하지는 않지만, 실용적인 상황에서 유용하게 사용됩니다.

따라서, 탐색할 하이퍼파라미터 범위가 작고 모든 조합을 시도할 여유가 있다면 `GridSearchCV`를, 탐색 범위가 넓고 시간적 제약이 있을 때는 `RandomizedSearchCV`를 선택하는 것이 좋습니다.